

DEALFLOW360 — COMPLETE PROJECT DESCRIPTION

1. What are we building?

DealFlow360 is an intelligent B2B sales operations platform.

Think of a company that sells products/services to other companies.

A salesperson gets a customer request:

"I need 100 laptops and a yearly software subscription. Give me your best price."

Normally, the salesperson has to manually:

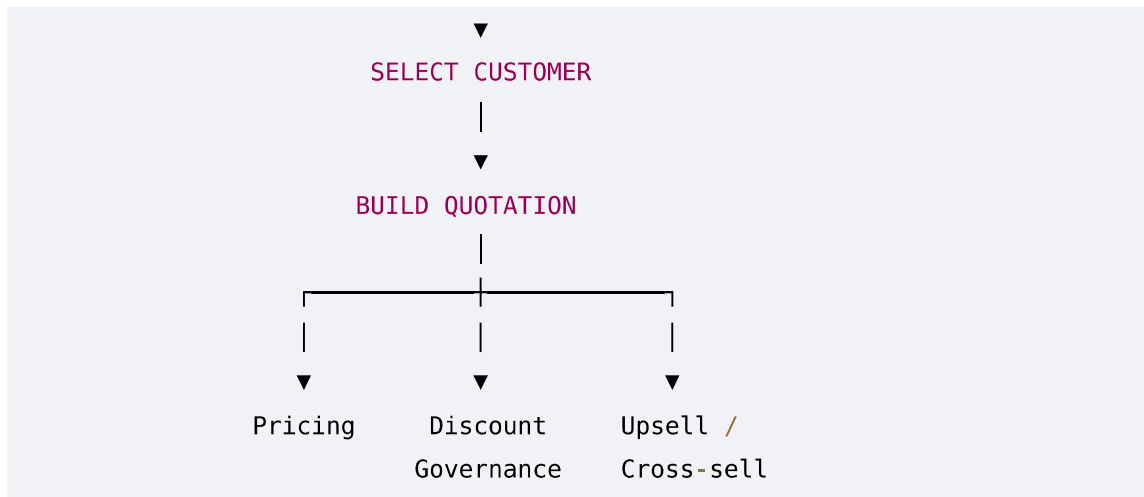
- Create a quotation
- Decide discounts
- Get manager approval
- Check inventory
- Figure out which warehouse should ship the products
- Handle products that are out of stock
- Create invoices
- Manage subscriptions
- Negotiate with the customer through email
- Track whether the deal is going well

Our system puts all of that into **one connected workflow**.

The hackathon specifically wants the system to go beyond a basic quote-to-invoice application and handle discount governance, inventory reality, hybrid billing, customer negotiation and deal monitoring.

2. The complete idea in one picture





The official quick test flow similarly goes from login → quotation → approval → upsell → warehouse split → hybrid billing → customer negotiation → payment.

3. The technology

We're keeping the technical architecture simple because you are three amateur developers using different AI coding assistants.

Frontend

React

Backend

Node.js
Express.js
JavaScript

Database

PostgreSQL

Database communication

Prisma ORM

Frontend ↔ Backend

REST API

Authentication

JWT

Docker

NO DOCKER

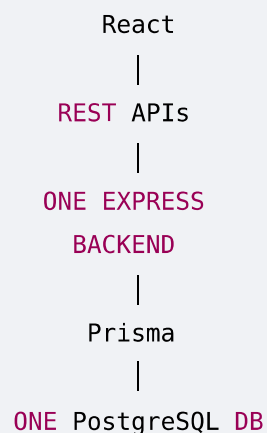
4. ONE backend

This is extremely important.

You are **not building three applications**.

You are building:

ONE DEALFLOW360 APPLICATION



All three members contribute to the same backend.

5. Three members

Member 1 — Yuvan Shankar B

Core Sales & Deal Management

Responsible for:

Authentication
Customers
Products
Pricing

Quotations
Discount Governance
Approval Workflow
Order Creation

Member 2 — Suryanarayanan D

Fulfillment & Billing

Responsible for:

Inventory
Warehouses
Warehouse Splitting
Fulfillment
Backorders
One-time Billing
Subscriptions
Proration
Invoices
Payments

Member 3 — Ashwin Perumal SR

Intelligence & Customer Experience

Responsible for:

Upsell / Cross-sell
Deal Health
Deal Rescue
What-If Simulator
Customer Portal
Negotiation

6. MEMBER 1 — CORE SALES

What is Member 1 doing?

Member 1 handles:

"How do we create a deal and get it approved?"

A salesperson logs in.

They select a customer.

They select products.

They enter quantities.

The system calculates pricing.

The salesperson applies a discount.

The system checks whether that discount is allowed.

If necessary, the system sends the quotation to the correct approver.

Once approved and accepted, it becomes an order.

7. Customer

Example:

ABC Technologies
Tier: GOLD
Contact: abc@example.com

Customer tier matters because discount permissions depend on it.

The PDF explicitly describes customer-tier discount ceilings such as Bronze, Silver and Gold.

8. Product

Example:

Laptop Pro
Category: Hardware
Price: ₹80,000
Type: One-time

Another:

Cloud Software

Category:

Subscription

Price:

₹10,000/month

Products are shared data.

Member 2 uses them for inventory and billing.

Member 3 uses them for recommendations.

9. Quotation

The salesperson builds:

Quotation #Q1001

Customer:

ABC Technologies

Laptop × 100

₹80,000

Cloud Software

₹10,000/month

The quotation contains:

Customer

Salesperson

Products

Quantities

Prices

Discounts

Tax

Total

Margin

Status

The frontend quotation builder is specifically expected to support products from Hardware, Services and Subscriptions, quantity adjustment, line/order discounts and live margin.

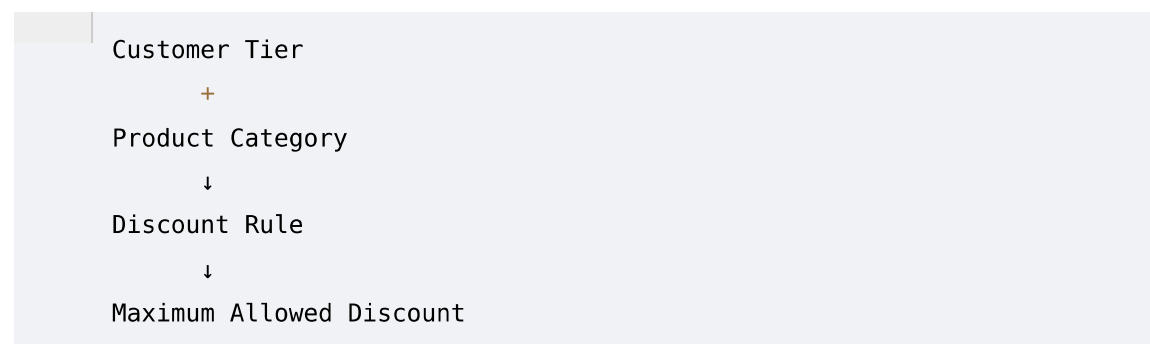
10. Discount Governance

This is a major part of the project.

The system doesn't simply allow the salesperson to enter:

```
50% discount
```

Instead:



Example:

```
Gold Customer

Hardware → 15% maximum
Service  → 10% maximum
```

Now:

```
Laptop
Discount = 12%
Allowed  = 15%

→ OK
```

But:

```
Setup Service
Discount = 18%
Allowed  = 10%
```

→ VIOLATION

The official example in the problem statement uses essentially this scenario and says the quotation should be flagged because the service line exceeds its stricter limit.

11. Blended Discount Risk

Sometimes individual violations are small.

Example:

```
Product A → 2% over
Product B → 3% over
Product C → 2% over
```

Each doesn't look terrible.

But together they represent significant discount leakage.

Therefore the system calculates a **blended risk score** across the quotation.

The purpose is to prevent a salesperson from staying just within or slightly beyond individual limits while giving away too much margin overall.

Important

The PDF specifies the concept of the blended score, but **doesn't prescribe a particular mathematical formula**.

So our implementation should use a transparent, documented rule rather than pretending there is an official formula.

12. Approval Workflow

Depending on the discount/risk:

```
Quotation
  ↓
Discount Check
  ↓
Does it require approval?
```

If no:

→ Continue

If yes:

Sales Manager

↓

Finance

only when the configured rules require the second level.

The backend should determine the approval chain.

Not React.

The problem statement explicitly requires configurable discount tiers and approval chains.

13. Member 2 starts here

Once the quotation becomes an accepted order:

Quotation

↓

Approved

↓

Accepted

↓

ORDER

Member 2 takes over.

14. MEMBER 2 — FULFILLMENT

Member 2 answers:

"Can we actually deliver what the customer ordered?"

Suppose:

Order:

100 Laptops

Inventory:

```
Warehouse A → 60
Warehouse B → 40
```

The system recommends:

```
Warehouse A → 60
Warehouse B → 40
```

The problem specifically requires warehouse splitting based on stock, with the ability for the user to manually override the suggested split.

15. Backorder

Suppose:

```
Required = 100

Warehouse A = 60
Warehouse B = 20
```

Available:

```
80
```

Then:

```
Fulfilled = 80
Backordered = 20
```

The system records the missing 20 rather than pretending everything shipped.

Later:

```
New stock arrives = 30
```

The system sees:

```
Backorder = 20
```

and can fulfill those 20.

16. Hybrid Billing

This is another major feature.

One order can contain:

Laptop
ONE-TIME
₹5,00,000

and:

Cloud Software
RECURRING
₹10,000/month

They remain part of **one order**, but billing handles them differently.

The PDF explicitly requires one order to mix one-time products and recurring subscription lines with correct billing schedules and proration.

17. Subscription

Example:

Cloud Software

₹10,000/month

Start:
1 September

Next billing:
1 October

If quantity changes halfway through the billing cycle, the system calculates the applicable partial-period amount.

The problem statement specifically calls for mid-cycle proration and subscription modification/cancellation behavior.

18. Invoice

The billing system produces:

Invoice #INV1001

Laptop

₹5,00,000

Subscription

₹10,000

Tax

₹X

Total

₹X

Payment then updates the invoice status.

Payment successful

↓

Invoice = PAID

19. MEMBER 3 — INTELLIGENCE

Member 3 sits **on top of the sales system**.

He doesn't replace Member 1 or 2.

He uses their data.

20. Upsell / Cross-sell

While the salesperson is building:

Laptop

the system can suggest:

Recommended :

Monitor

Warranty

Installation Service

The suggestion can use:

- Co-purchase history
- Promotions

- Product relationships
- Margin

The official frontend specification calls for ranked suggestions, margin delta and promotion information.

21. Deal Health Monitoring

This asks:

"How healthy is this deal?"

The system looks at signals such as:

```
Discount
Margin
Quote inactivity
Negotiation
Delivery status
```

Example:

```
Deal #Q1001

Discount: 20%
Margin: 7%
Inactive: 9 days

⚠ HIGH RISK
```

The dashboard should show stalled deals, discount anomalies and delivery-promise slippage.

22. Deal Rescue Engine

This goes one step further.

Deal Health:

"Something is wrong."

Deal Rescue:

"What should the salesperson do?"

Example:

HIGH RISK

Reason:

18% discount

Low margin

Customer inactive for 8 days

Suggested rescue:

→ Reduce discount to 12%

→ Offer free installation

→ Add a monitor bundle

→ Contact customer

23. What-If Deal Simulator ★

This is one of our strongest demo features.

The salesperson asks:

"What happens if I increase the discount from 10% to 15%?"

The system temporarily calculates:

Current

Discount: 10%

Margin: 18%

Approval: No

What-If

Discount: 15%

Margin: 13%

Approval: Required

Nothing is actually changed in the quotation until the user chooses to apply it.

And the simulator should reuse the **same pricing/discount/approval services** as the real quotation system.

That gives us:



One business rule.

Two situations.

24. Customer Portal

The customer gets a separate interface.

Not the internal sales screen.

The customer sees:

Quotation #	Q1001
Laptop ×	100
	₹80,000
Discount	
	12%
Total	
	₹X

They can:

Accept
Request change
Comment on a line

```
Counter the discount
Confirm quotation
```

The problem statement specifically requires a real restricted customer-facing view rather than simply relabeling an internal screen.

25. Negotiation

Customer says:

```
"Can you give me 15% instead?"
```

The system does:

```
Customer
  ↓
Negotiation Request
  ↓
Quotation
  ↓
Discount Rule Check
  ↓
Approval Required?
```

If the new terms exceed the threshold:

```
→ Approval again
```

The official flow explicitly requires a quotation to re-enter approval if customer negotiation pushes the final terms beyond approval thresholds.

26. AI's actual role

We **should use AI**, but we should not make AI responsible for the core calculations.

Normal backend logic

```
Discount calculation
Inventory
Approval
Pricing
Billing
Proration
```

should be deterministic application logic.

The problem statement itself says core business rules such as approval routing, discount governance, warehouse splitting and billing proration must be implemented in application logic, not faked for the demo.

AI can help with:

```
Recommendations
Deal explanations
Deal rescue suggestions
Natural-language summaries
Negotiation insights
```

So:

```
BACKEND
"Margin = 8%"

↓

AI

"This deal is at risk because
the high discount has reduced
margin to 8%."
```

That's much safer than:

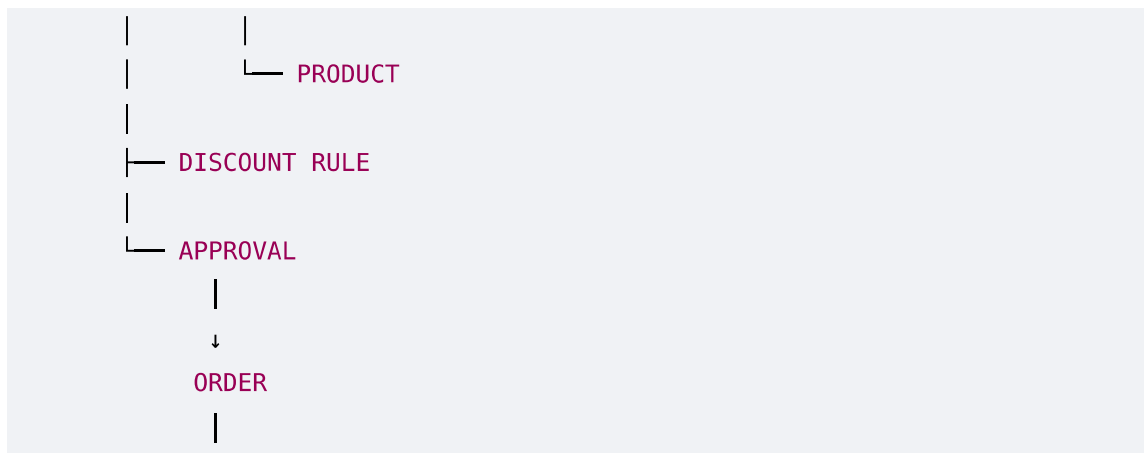
```
"AI decided that margin is 8%."
```

27. Shared database

The three AIs must work with **one Prisma schema**.

Conceptually:

```
USER
|
CUSTOMER
|
QUOTATION
|
├── QUOTATION ITEMS
```



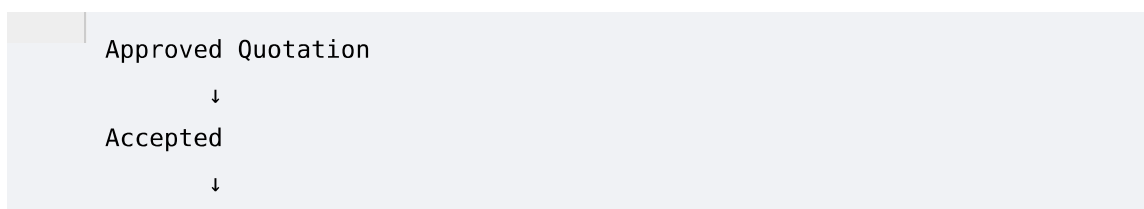
And Member 3 reads from these same entities for intelligence.

28. Suggested backend structure



29. How the three modules communicate

Member 1 → Member 2



Order



Member 2

Member 1 → Member 3

Quotation

Discount

Margin

Customer

Products

Status



Member 3

Member 2 → Member 3

Inventory

Fulfillment

Backorders

Delivery status



Member 3

This lets Member 3 detect things such as:

"This deal is at risk because 40% of the order is currently backordered."

30. The three AIs MUST follow these rules

Give this to **all three AIs**.

1. Use the existing DealFlow360 repository.

2. Use **ONE** Express backend.

3. Use **ONE** PostgreSQL database.

4. Use Prisma **as** the database **ORM**.

5. Do **NOT** create another backend.

6. Do **NOT** create another database.
 7. Do **NOT** create duplicate Customer, Product, Quotation, Order or User entities.
 8. Inspect the existing Prisma schema before modifying it.
-

31. The entire product in one example

Imagine **ABC Technologies** wants:

```
100 Laptops
100 Monitors
Cloud Software Subscription
```

Step 1 — Salesperson

Creates quotation.

```
100 Laptop
100 Monitor
Subscription
```

Step 2 — Discount

Salesperson requests:

```
Laptop → 12%
Monitor → 15%
```

System checks:

```
Laptop allowed → 15%
Monitor allowed → 10%
```

Monitor violates the rule.

```
→ Approval required
```

Step 3 — Upsell

System suggests:

```
Warranty
```

Salesperson adds it.

Margin updates.

Step 4 — Manager

Manager sees:

```
Discount Risk
Approval Chain
Quotation
Margin
```

Approves.

Step 5 — Customer

Customer opens portal.

Says:

```
"Can you make the monitor discount 17%?"
```

System checks again.

17% is outside the configured limit.

```
→ Approval again
```

Step 6 — Order

Customer confirms.

```
Quotation
↓
Order
```

Step 7 — Warehouse

Inventory:

```
Warehouse A → 60 laptops
Warehouse B → 40 laptops
```

System recommends:

```
A → 60
B → 40
```

Step 8 — Backorder

Suppose only 80 monitors exist.

Requested → 100
Available → 80
Backorder → 20

Step 9 — Billing

Order contains:

Laptop → One-time
Monitor → One-time
Cloud Software → Recurring

System creates appropriate billing records.

Step 10 — Deal Health

Dashboard notices:

High discount
+
Backordered products
+
Customer negotiation

Therefore:

⚠ HIGH RISK DEAL

Step 11 — Deal Rescue

System suggests:

"Consider offering free installation instead of increasing the discount."

Step 12 — Payment

Customer pays.

Invoice → PAID

And now the entire journey is:



That is DealFlow360. The PDF's stated goal is essentially this complete quotation-to-cash experience, with the business logic and operational problems handled inside one connected system.

The simplest way to remember the three members

Yuvan creates and gets the deal approved.

Suryan delivers and bills the deal.

Ashwin watches, improves and helps save the deal.

And **all three are working on the same backend and the same database.**